NGINX Web Server

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Floriano Scioscia

Gabriele Colapinto

May 19, 2026

License

Contents

1 Nginx: An Event-Driven Alternative to the Apache Web Server

Nginx (pronounced "engine-x") is an open-source web server that was developed specifically to address the performance limitations of traditional servers like Apache, particularly in handling a high number of concurrent connections. It has gained immense popularity for its high performance, resource efficiency, and powerful capabilities as a reverse proxy and load balancer. Its architecture also enables key operational advantages, such as the ability to perform configuration reloads and "graceful stops" without dropping active connections.

1.1 A Fundamentally Different Architecture

The primary difference between Nginx and Apache lies in their core architectural models. Nginx employs an **asynchronous, event-based, non-blocking architecture**. It operates with a master process that is responsible for reading and validating the configuration and managing the lifecycle of a small, fixed number of worker processes. These workers, often configured to match the number of CPU cores, handle the actual connections and can each manage thousands of concurrent requests simultaneously.

The responsibilities of the master process include:

- reading and validating the configuration file;
- starting, stopping, and maintaining worker processes;
- reloading the configuration and reconfiguring the server without interrupting active connections or causing service downtime.

Worker processes are the components that actually handle client requests. Instead of creating a dedicated process or thread for every incoming connection, each worker uses an event loop capable of managing thousands of concurrent connections within a single process. This design dramatically reduces memory usage and context-switching overhead.

A central component of this architecture is the shared **listen socket**. A socket is an operating system abstraction representing one endpoint of a network communication channel. The listen socket is created by the master process and shared among all worker processes. It remains bound to a specific IP address and port (for example, port 80 for HTTP or port 443 for HTTPS) and waits for incoming client connections.

When a new client connection arrives, the operating system notifies one of the worker processes that the listen socket is ready to accept the connection. This mechanism avoids the need for every worker to continuously poll for new requests and allows the operating system to efficiently distribute incoming connections among the available workers.

Nginx relies on operating system dependent event notification mechanisms such as:

- `epoll` on Linux;
- `kqueue` on BSD-based systems and macOS;
- `select` or `poll` on older Unix systems.

These mechanisms allow a worker process to monitor thousands of sockets simultaneously and react only when a socket becomes ready for reading or writing. This approach is significantly more efficient than continuously checking every connection.

For example, suppose a web server receives 10,000 simultaneous client connections. In a traditional process-per-connection architecture, the server might need thousands of threads or processes, consuming large amounts of memory and CPU time due to context switching. In contrast, Nginx may use only a small number of worker processes (for example, four workers on a quad-core CPU). Each worker can asynchronously manage thousands of active connections using the operating system event notification system. When data becomes available on a socket, the OS wakes up the appropriate worker, which processes the request without blocking other connections.

This architecture is in stark contrast to Apache's older synchronous model, which traditionally assigned a new process or thread to each connection, leading to high memory consumption and performance degradation under heavy load. The direct impact of Nginx's architecture is significant: it uses substantially less memory and delivers superior performance at high levels of concurrency, making it an excellent choice for modern, high-traffic web applications.

1.1.1 Event Loop, Session Management, and User Isolation

One of the most important aspects of the Nginx architecture is that a single worker process can handle thousands of concurrent client connections simultaneously using an event-driven model. This may initially raise concerns regarding authentication, session management, and user isolation, especially when authenticated and unauthenticated users are being served concurrently by the same worker process.

In reality, the event loop mechanism used by Nginx is responsible only for managing **network I/O events** and does not directly manage user authentication or application state. Each worker process maintains an internal event queue containing tasks generated by many different users and connections. Consequently, a single CPU core may process requests belonging to thousands of different clients in rapid succession.

Unlike traditional thread-per-request architectures, Nginx does not dedicate a specific thread or process to each user. Instead, worker processes asynchronously react to events generated by sockets associated with different clients. The worker simply determines which socket is ready for reading or writing and processes the corresponding request.

For example, a worker process may internally alternate between:

- a request from an authenticated user;
- a request from an anonymous client;
- an HTTPS handshake;
- a WebSocket event;

- a static file request.

This design remains safe because each HTTP request carries all the information necessary to identify the client, typically through:

- cookies;
- session identifiers;
- authentication tokens;
- authorization headers.

In a typical PHP-based web application, authentication is not handled directly by Nginx. Instead, Nginx forwards dynamic requests to an application server such as PHP-FPM. The application layer is responsible for managing sessions and authentication state. For instance, when a client sends a request containing:

```
Cookie: PHPSESSID=abc123
```

PHP uses the session identifier to retrieve the corresponding session data from a dedicated session storage system. Depending on the server configuration, session data may be stored:

- in files on disk;
- in memory databases such as Redis or Memcached;
- inside relational or NoSQL databases.

The stored session may contain information such as:

- user identifier;
- authentication status;
- roles and permissions;
- shopping cart contents;
- CSRF tokens (Cross-Site Request Forgery, a unique, secret, and unpredictable value generated by a server and given to a user's browser. It prevents hackers from executing unauthorized actions on your behalf by ensuring that any form submission or API request genuinely originated from your trusted session).

Therefore, the worker process itself does not permanently store the authentication state of every user in its own memory. Instead, it temporarily handles requests and delegates session management to specialized application-layer components. Internally, each active connection handled by a worker process is associated with lightweight data structures containing:

- socket descriptors;
- network buffers;
- connection state;
- timers and event metadata.

These structures are isolated for each connection and are managed independently by the operating system and the Nginx event loop. The operating system further guarantees process-level memory isolation, preventing one request from accidentally accessing another user's data.

Security therefore does not depend on assigning one worker per user, but rather on:

- isolated connection contexts;
- unique session identifiers;
- proper session management;
- operating system memory protection;
- application-layer authentication mechanisms.

As a result, a small number of Nginx worker processes can efficiently and safely manage tens of thousands of concurrent users while maintaining strict separation between user sessions and authentication states.

1.2 The Nginx Configuration Model

Like Apache, Nginx uses a text-based configuration file composed of directives. The main configuration file is typically called `nginx.conf`. Unlike Apache, however, Nginx organizes directives into different hierarchical **contexts**, allowing configuration rules to be applied at different levels of the web server architecture. Nginx directives are divided into two categories:

- **Simple directives**: consist of a directive name followed by parameters separated by spaces and terminated by a semicolon. For example:

```
worker_processes 4;
```

- **Block directives**: consist of a directive name followed by a block of additional directives enclosed in curly braces. These blocks define a configuration context. For example:

```
server {
        listen 80;
        server_name example.com;
}
```

The configuration hierarchy is organized into three main contexts:

1. `http`: the top-level context for the entire web server instance. It contains global HTTP configuration directives and one or more `server` blocks.
2. `server`: defines a specific virtual host. A server block specifies the IP address and port on which the server listens, together with the domain names associated with that virtual host.
3. `location`: defines configuration rules for a specific set of resources or URI paths inside a virtual host. Requests can be matched using prefixes or regular expressions. Multiple nested `location` contexts are allowed.

Nginx processes requests hierarchically according to the following order:

$$\texttt{http} \rightarrow \texttt{server} \rightarrow \texttt{location}$$

First, Nginx determines which `server` block should process the incoming request based on:

- the listening port;
- the IP address;
- the requested domain name specified in the `Host` header.

The configuration file may therefore contain multiple `server` blocks implementing different virtual hosts on the same physical machine. Once the correct server block has been selected, Nginx evaluates the requested URI against the available `location` directives. The goal is to determine which configuration block most specifically matches the request. For matching requests, the URI is combined with the path specified by the `root` directive to construct the path of the requested resource on the local filesystem. For example:

```
location /images/ {
        root /var/www/html;
}
```

A request for:

```
/images/logo.png
```

will correspond to the following file path on disk:

```
/var/www/html/images/logo.png
```

When multiple `location` blocks match a request, Nginx selects the **most specific match**, generally corresponding to the location block with the longest matching prefix. This mechanism allows administrators to define both general rules and highly specialized configurations for specific resources or application paths. Directives defined in more specific contexts override directives inherited from more general contexts. Consequently, a directive specified inside a `location` block takes precedence over the same directive defined at the `server` or `http` level.

This hierarchical structure is illustrated in the example below:

```
http {
    server {
        listen 80;
        server_name www.example.com;

        location / {
            root /data/www;
        }

        location /images/ {
            root /data;
        }
    }
}
```

This example shows a top-level `http` block containing a `server` block for a virtual host. Inside, two `location` blocks define different document roots for different URI paths. Given this configuration file, if a user requests the resource at the following URI:

```
http://www.example.com/images/example.png
```

Nginx chooses the second location because it has a longer prefix and is more specific (`/images` is longer than `/`) and returns to the client the file stored in the following path on th disk:

```
/data/images/example.png
```

1.3 Common Nginx Use Cases

The following examples demonstrate two of the most common Nginx configurations: virtual hosting and reverse proxying.

1.3.1 Virtual Host Configuration

```
server {
        listen 4000;
        server_name www.example.com;

        location / {
                root /usr/share/nginx/html/vhost;
                index index.html index.htm;
        }
}
```

This `server` block configures a complete virtual host. The directive:

```
listen 4000;
```

instructs Nginx to accept incoming TCP connections on port 4000.

The directive:

```
server_name www.example.com;
```

specifies the hostname associated with this virtual host. Nginx uses the value contained in the HTTP `Host` header to determine which `server` block should process the request.

To test the configuration locally, the hostname must be mapped to an IP address through the operating system hosts file. For example:

```
127.0.0.1 www.example.com
```

This manual mapping acts as a local DNS resolution rule, associating the hostname `www.example.com` with the loopback address `127.0.0.1`. Consequently, requests sent to:

```
http://www.example.com:4000
```

will reach the local Nginx instance.

Inside the `location` block, the directive:

```
root /usr/share/nginx/html/vhost;
```

defines the root directory used to retrieve static resources from the filesystem.

The `index` directive specifies the default file that should be served when the client requests a directory instead of a specific file:

```
index index.html index.htm;
```

The directive accepts multiple filenames because Nginx tests them sequentially and returns the first existing file to the client. In this case:

1. Nginx first searches for `index.html`;
2. if the file does not exist, it searches for `index.htm`.

Therefore, the directive does not indicate multiple simultaneous files, but rather an ordered list of fallback default resources.

1.3.2 Reverse Proxy Configuration

```
server {
        listen 5000;
        server_name localhost;

        location / {
                proxy_pass http://host.docker.internal:80;
        }

        location ~ \.(gif|jpg|png)$ {
                root /usr/share/nginx/html/proxy;
        }
}
```

This configuration demonstrates the use of Nginx as a reverse proxy server.

The directive:

```
proxy_pass http://host.docker.internal:80;
```

forwards incoming requests to another web server running on a different host or container. In this example, the backend server listens on port 80.

The hostname:

```
host.docker.internal
```

is a special DNS name provided by Docker. It automatically resolves to the internal IP address of the host machine from inside a Docker container. This mechanism allows a containerized Nginx instance to communicate with services running on the host operating system. Consequently, when a client accesses:

```
http://localhost:5000
```

Nginx receives the request and transparently forwards it to:

```
http://host.docker.internal:80
```

The client therefore interacts only with Nginx, while the actual content may be generated by another backend application server.

The second `location` block:

```
location ~ \.(gif|jpg|png)$
```

uses the symbol `~` to indicate that the URI should be matched using a regular expression. The regular expression:

```
\.(gif|jpg|png)$
```

matches all URIs ending with:

- `.gif`
- `.jpg`
- `.png`

For example:

- /images/photo.jpg
- /icons/logo.png

When such a request is detected, Nginx serves the resource directly from the local filesystem instead of forwarding the request to the backend server. The directive:

```
root /usr/share/nginx/html/proxy;
```

specifies an absolute filesystem path used as the root directory for static resources handled by this location block. For example, the request:

```
/images/photo.jpg
```

will correspond to the following filesystem path:

```
/usr/share/nginx/html/proxy/images/photo.jpg
```

This mechanism is extremely efficient because Nginx can serve static files such as images directly, without involving the backend application server. As a result:

- backend load is reduced;
- response times improve;
- memory and CPU usage decrease;
- static content delivery becomes highly scalable.

This architectural pattern is commonly used in modern web infrastructures, where Nginx acts simultaneously as:

- reverse proxy;
- load balancer;
- SSL/TLS terminator;
- static content server.

While their syntax and architecture differ, both Apache and Nginx provide powerful tools for implementing key web architecture patterns. One of the most critical of these is load balancing, a technique essential for building scalable and reliable services.

2 Nginx Installation and Service Management

Nginx is available for most modern operating systems and can be installed using either precompiled packages or by compiling the source code manually. The installation process and management commands are designed to be lightweight and efficient, reflecting the overall philosophy of the web server.

2.1 Installing Nginx

On Linux systems, Nginx is commonly installed through the package manager provided by the distribution. Prebuilt packages are available in formats such as:

- RPM packages for Red Hat, CentOS, Fedora, and related distributions;
- DEB packages for Debian, Ubuntu, and related distributions.

The official installation instructions for supported Linux distributions can be found on the official Nginx website: `https://nginx.org/en/linux_packages.html`.

Installing from precompiled packages is generally the simplest and most reliable approach because:

- dependencies are automatically resolved;
- updates can be managed through the package manager;
- the installation follows the conventions of the operating system.

Alternatively, Nginx can be compiled directly from source code. This approach is useful when:

- custom modules are required;
- specific compile-time options must be enabled or disabled;
- experimental versions need to be tested;
- the administrator wants maximum control over the installation.

Compilation from source typically follows the traditional Unix workflow:

```
./configure
make
make install
```

The `configure` script checks system dependencies and generates the appropriate build configuration, while `make` compiles the source code. The official documentation for compilation options is available at `https://nginx.org/en/docs/configure.html`.

By default, the Nginx executable is installed in:

```
/usr/sbin/nginx
```

However, the installation path can be modified using specific compilation options during the `configure` phase. The default location of the main configuration file is:

```
/etc/nginx/nginx.conf
```

This file represents the main entry point of the Nginx configuration hierarchy and may include additional configuration files using the `include` directive.

For Microsoft Windows systems, Nginx is distributed as a precompiled binary package. Although functional, the Windows version is generally considered less optimized than the Unix/Linux implementation because Nginx was originally designed around Unix event-driven mechanisms such as `epoll`.

2.2 Starting and Managing the Nginx Service

The Nginx daemon can be started directly by executing the binary:

```
/usr/sbin/nginx
```

Starting the server typically requires root privileges because Nginx often binds to privileged ports such as:

- port 80 for HTTP;
- port 443 for HTTPS.

The executable supports several optional parameters used to control the daemon during runtime.

2.2.1 Stopping the Server

To immediately terminate the Nginx server, the following command can be used:

```
/usr/sbin/nginx -s stop
```

This operation forces the server to terminate quickly and closes active connections abruptly.

2.2.2 Graceful Shutdown

A safer alternative is the graceful shutdown command:

```
/usr/sbin/nginx -s quit
```

In this case, Nginx stops accepting new client connections but allows already active connections to complete before terminating worker processes.

A graceful stop is particularly important in production environments because it prevents:

- interrupted downloads;
- incomplete HTTP responses;
- abrupt client disconnections;
- loss of in-progress requests.

This mechanism is possible because the master process coordinates worker termination while monitoring active connections.

2.2.3 Reloading the Configuration

One of the most powerful features of Nginx is the ability to reload the configuration without interrupting service availability:

```
/usr/sbin/nginx -s reload
```

When this command is executed:

1. the master process rereads the configuration file;
2. the configuration syntax is validated;
3. new worker processes are spawned with the updated configuration;
4. old workers continue serving existing connections;
5. once active requests terminate, the old workers are gracefully stopped.

This architecture allows configuration changes to be applied "on the fly" without downtime, which is one of the reasons why Nginx is widely adopted in high-availability infrastructures.

2.2.4 Testing the Installation

Once the server has been started successfully, its operation can be verified by opening a web browser and navigating to:

```
http://localhost
```

If the installation and configuration are correct, Nginx returns a default welcome page or the configured website content (see figure 2.1).

The term `localhost` refers to the local machine itself and typically resolves to the loopback IP address:

Figure 2.1: Nginx welcome message

```
127.0.0.1
```

This test confirms that:

- the Nginx daemon is running;
- the server is listening on the expected port;
- the network configuration is functioning correctly;
- the HTTP request-response cycle operates properly.

3 Containerization with Docker and Docker Compose

Containerizing NGINX ensures environment parity across development and production, simplifying deployment workflows.

3.1 Docker Compose Service (docker-compose.yml)

To manage the deployment and persistence of these containers, `docker-compose.yml` defines the service parameters, specifically port mapping and volume externalization:

```
version: '3'
services:
  nginx:
        restart: always
        image: nginx:latest
        container_name: web
        ports:
          - "8080:80"   # Maps Host Port 8080 to Container Port 80
          - "5000:5000"
          - "4000:4000"
        volumes:
          - nginx_root:/usr/share/nginx/html
          - nginx_conf:/etc/nginx/conf.d
volumes:
  nginx_root:
        external: true
  nginx_conf:
        external: true
```

The `ports` mapping (e.g., `8080:80`) is crucial: it allows external traffic on the host's port 8080 to be routed into the container's standard web port. This ensures the service is accessible while maintaining isolation.

After creating the `docker-compose.yml` file, we can run the command `docker compose up -d` to start the Docker application and all the services defined in the YAML file.

3.2 Custom NGINX Dockerfile

To build a custom image, a `Dockerfile` is used to package the configuration and static assets:

```
FROM nginx:latest
```

```
# Copy local configs and static content into the image
COPY ./nginx_conf/. /etc/nginx/conf.d/
COPY ./nginx_root/. /usr/share/nginx/html/

# Define accessible container ports
EXPOSE 80 4000 5000
```

After creating this `Dockerfile`, we can build the container image using the command: `docker build –t nginx-test`. Then, we can start the container using the command `docker run -d –p 8080:80 –p 4000:4000 –p 5000:5000 --name web nginx-test` and check if all files are available using the command `docker exec –it web /bin/bash`.

4 A Deeper Dive into Load Balancing

Load balancing is the process of distributing incoming tasks or network traffic across a set of resources, such as a cluster of servers. While reverse proxying is the *mechanism*, load balancing is the strategic *application* of that mechanism to achieve high availability and scalability—two pillars of modern web architecture. The primary goal is to optimize response time, maximize throughput, and prevent any single resource from becoming overloaded. Both Apache and Nginx can function as powerful software load balancers, making this a foundational strategy for building resilient web services.

4.1 Core Concepts and Scheduling Algorithms

Load balancing algorithms determine how tasks are distributed among the available servers. They can be broadly categorized into two types.

1. **Static vs. Dynamic Algorithms**: Static algorithms distribute tasks based on a fixed set of rules, without considering the current state or load of the servers. Dynamic algorithms, in contrast, are more sophisticated and adapt to the real-time load of each server, potentially moving tasks from an overloaded node to an underloaded one.
2. **Common Scheduling Algorithms**: Several common static algorithms are used for their simplicity and effectiveness:
    - **Round-Robin**: Requests are distributed to servers in a sequential, rotating order. The first request goes to the first server, the second to the second, and so on, cycling back to the beginning after the last server is reached.
    - **Randomized Static**: Tasks are assigned to servers at random. With a large number of requests, this method achieves a reasonably even distribution across the server pool.

4.2 Implementation Strategies

For internet services, load balancing can be implemented using two main strategies.

1. **DNS-Based Load Balancing**: This technique, often called Round-Robin DNS, associates a single domain name with multiple IP addresses. When a client requests the domain, the DNS server returns the IP addresses in a rotating order. While simple, this method can be unreliable due to DNS caching.
2. **Server-Side Load Balancers**: This is the more common and robust approach. A dedicated hardware appliance or a software program (such as Apache or Nginx in reverse proxy mode) acts as a single entry point for all client traffic. This load balancer receives requests and intelligently forwards them to one of several backend servers, making the backend infrastructure transparent to the client.

4.3 The Challenge of Persistence

In a load-balanced environment, a significant challenge arises with managing user sessions.

1. **The Problem**: If a user's session data (e.g., items in a shopping cart) is stored locally on one backend server, a problem occurs if the load balancer sends their next request to a different server. That new server will have no knowledge of the user's session, leading to a broken user experience.
2. **The Solution ("Stickiness")**: The solution is **persistence**, often called "stickiness." This is a technique where the load balancer ensures that all requests from a single user's session are consistently routed to the same backend server. This can be achieved by tracking the client's IP address or other identifiers, thereby maintaining session state.

Managing state is a key consideration when designing any load-balanced architecture, as it directly impacts both user experience and system reliability.

4.4 Load Balancing in NGINX

Load balancing is implemented to maximize throughput, reduce latency, and ensure fault tolerance. Rather than "scaling up" with expensive hardware, NGINX facilitates a "scale-out" architecture, distributing traffic across a fleet of identical, standardized server instances. Architects use the `upstream` context to group backend instances. Within this context, specific policies can be applied:

Policy Name	**Directive**	**Mechanism**	**Use Case / Benefit**
Round Robin	*(Default)*	Requests are distributed sequentially.	Standard distribution for identical backends.
Least Connected	`least_conn;`	Targets the server with the lowest active connections.	Prevents overloading busy application servers.
IP Hash	`ip_hash;`	Maps client IP to a specific server via a hash function.	Ensures session persistence for stateful apps.

4.5 Load Balancer implementation in NGINX

```
http {
    upstream myapp1 {
        # Here we can use a policy directive from
        # the table. Leaving this empty implements
        # the round robin policy by default.
        ip_hash; # Ensures session persistence
        # least_conn; # Prevents server overloading

        server srv1.example.com;
        server srv2.example.com;
        server srv3.example.com;
    }
```

```
        server {
            listen 80;
            location / {
                proxy_pass http://myapp1;
            }
        }
    }
```

By replicating server instances with identical configurations, NGINX provides high reliability. If one node fails, the load balancer continues to respond using the remaining healthy instances, ensuring seamless failover.

5 Comparative Performance Analysis: NGINX vs. Apache HTTP Server

The competition between NGINX and Apache 2.4 has defined modern web performance standards. While Apache has introduced a new architecture to improve its efficiency, key differences remain based on the application's resource profile:

- **Concurrency:** NGINX is engineered to handle extreme levels of simultaneous connections (concurrency) far more efficiently than Apache.
- **Static vs. Dynamic:** NGINX is significantly faster at serving static resources (images, video, music). Conversely, Apache has historical strengths in handling dynamic content and complex business logic.

5.1 The Ideal Hybrid Configuration

The industry-standard architectural solution is to combine both systems to leverage their respective strengths. In this hybrid model, NGINX serves as the front-facing server:

1. **NGINX** handles all requests for static assets directly from the file system.
2. **NGINX** acts as a reverse proxy for dynamic requests, forwarding them to an **Apache** instance running on the internal network to process business logic.

Choosing between these systems—or implementing them in tandem—should be based on the nature of the application: static-heavy workloads favor NGINX, while applications requiring extensive dynamic data generation via AJAX or complex logic may still utilize Apache as the backend engine.